

An Introduction to Agents and Agentic AI 🤖

Day – 12

What is an AI Agent?

An **AI agent** is a system that can perceive its environment and take actions to achieve specific goals. Think of it like a self-sufficient worker in a digital space.

- **Core Components:** Explain the basic loop:
 1. **Perception:** The agent 'sees' its environment (e.g., receives an email, gets a user request, reads data).
 2. **Thinking/Planning:** The agent 'thinks' about its goal and decides what to do next.
 3. **Action:** The agent 'acts' to carry out its plan (e.g., sends a reply, searches the web, writes code).
 - **PEAS Framework:** Introduce this simple concept to define any agent:
 - **Performance Measure:** How do we know if the agent is doing a good job? (e.g., speed, accuracy, cost).
 - **Environment:** Where does the agent operate? (e.g., a computer's file system, the internet, a specific software).
 - **Actuators:** How does the agent take action? (e.g., keyboard strokes, API calls, mouse clicks).
 - **Sensors:** How does the agent perceive its environment? (e.g., camera input, text from a prompt, file data).
-

What is Agentic AI?

Agentic AI is the implementation of this agent concept using modern AI, particularly Large Language Models (LLMs) like GPT-4. It's an architecture that allows an AI to autonomously plan and execute multi-step tasks.

- **The Goal:** The main goal of agentic AI is to complete complex, long-term objectives with minimal human intervention. Instead of you giving ten small commands, you give one big goal.
- **Key Architectural Components:**
 1. **Core Brain (LLM):** This is the central reasoning engine. It takes the goal, breaks it down into smaller steps, and decides what to do.

2. **Memory:** The agent needs to remember what it has done, what it has learned, and what the original goal was. This can be short-term (for the current task) or long-term (learning from past tasks).
3. **Tools:** These are the skills the agent can use. An agent isn't just a brain; it has hands. Tools can include:
 - Web search access
 - Code execution capabilities
 - Access to specific databases or APIs (e.g., your company's sales data)
 - Ability to use other software

Practical Example: The Automated Research Assistant

To make it concrete, describe a practical example.

- The Goal (given by a human): "Create a summary report on the latest trends in renewable energy in India, including key companies, recent investments, and government policies. Save it as a PDF."
- How an Agentic AI Would Work:
 1. **Plan:** The LLM brain breaks the goal down: "First, I need to search for recent renewable energy trends in India. Then, I'll identify top companies. Next, I'll look for investment news and government policy documents. Finally, I will synthesize all this information and write a report."
 2. **Action (Using Tools):**
 - Uses the web search tool to find articles and reports.
 - Reads the search results and saves key information to its memory.
 - Performs new searches based on what it finds (e.g., searches for a specific company it discovered).
 - Once all information is gathered, it uses its internal knowledge to write the report.
 3. **Final Output:** The agent creates a `summary_report.pdf` and notifies the user that the task is complete.

This is the agent's only tool. It "checks" the weather.

```
def get_weather():
```

```
    """A simple tool that pretends to check the weather."""
```

```
print(" -> ACTION: Checking the weather...")

# In a real program, this would connect to a weather service.

# For our example, we'll just return a fixed answer.

return "rainy"
```

1. Define the Agent's Goal

```
goal = "Figure out what to wear today."

print(f"🎯 AGENT'S GOAL: {goal}\n")
```

2. The Agent's Memory (what it knows)

We'll use a dictionary, which is a standard Python feature.

```
memory = {

    "weather": "",

    "what_to_wear": ""

}
```

3. The Agent's Thinking and Acting Loop

```
print("--- AGENT STARTING ---")
```

Step 1: Check what is needed

```
if memory["weather"] == "":

    print("😬 PLAN: I need to know the weather.")

    # Use the tool to get the weather

    weather_today = get_weather()

    # Remember the result

    memory["weather"] = weather_today

    print(f"🧠 MEMORY UPDATED: Weather is '{memory['weather']}'\n")
```

Step 2: Make a decision based on memory

```
if memory["weather"] != "" and memory["what_to_wear"] == "":

    print("😬 PLAN: I know the weather, now I need to decide what to wear.")

    if memory["weather"] == "rainy":

        # Make a decision

        decision = "a raincoat and boots"
```

```
print(f" -> ACTION: Decided to wear {decision}.")

# Remember the decision

memory["what_to_wear"] = decision

print(f"🧠 MEMORY UPDATED: I will wear '{memory['what_to_wear']}'\n")

# --- Final Result ---

print("--- AGENT FINISHED ---")

print(f"✅ GOAL ACHIEVED: The agent decided to wear: {memory['what_to_wear']}.")
```

How It Works:

This code demonstrates the core agent loop without any complexity:

1. **Goal:** The agent has a clear objective.
2. **Memory:** A simple Python dictionary holds what the agent knows.
3. **Plan/Act:** The if statements act as the agent's "brain." It checks its memory and decides what to do next.
4. **Tool Use:** The `get_weather()` function is the agent's "hand," allowing it to interact with its (pretend) environment.
5. **Update Memory:** After each action, the agent saves what it learned, which helps it make the next decision.